DISTRIBUTED SYSTEMS

Complete Reference Guide

Deep Dive into Architectures · Algorithms · Protocols · Real-World Examples

Topics Covered

Fundamentals & Architecture • Communication Models
Time & Ordering • Consistency & Replication
Consensus Algorithms • Fault Tolerance
Distributed Transactions • CAP & PACELC Theorems
Load Balancing & Scalability • Distributed Storage
Microservices & Service Mesh • Observability & Monitoring

Table of Contents

CHAPTER 1

Fundamentals of Distributed Systems

Definition, Properties, Challenges & Types

1.1 What Is a Distributed System?

A **distributed system** is a collection of autonomous computing elements (nodes/processes) that appears to its users as a single coherent system. Nodes communicate and coordinate by passing **messages** over a network. Neither shared memory nor a global clock is assumed.

■ Example: Google Search

When you type a query, thousands of servers across multiple data centres collaborate — crawlers, indexers, ranking engines, cache layers — yet you see one unified result page within milliseconds. That invisibility of distribution is the core goal.

1.2 Key Properties

Property	Explanation
Concurrency	Multiple nodes execute simultaneously; events at different nodes happen at the same time.
No Global Clock	No single authoritative clock; nodes must agree on ordering via logical mechanisms.
Partial Failures	A subset of nodes or links can fail while the rest continue to operate.
Message Passing	All coordination is via asynchronous or synchronous messages; no shared memory.
Openness	Standard protocols let heterogeneous systems interoperate and be extended.
Transparency	Hide distribution: access, location, migration, replication, failure, concurrency.

1.3 Core Design Challenges

■ Network unreliability

Packets can be lost, reordered, or duplicated. TCP gives reliability at cost of latency.

■ Latency variability

LAN ≈ 0.1 ms; cross-continent ≈ 150 ms; disk seek ≈ 5 ms. Design must account for tail latency.

■ Bandwidth limits
Sending full datasets repeatedly is costly; use deltas, compression, and smart caching.

■ Security
An open network means adversaries. Authentication, authorisation, and encryption are mandatory.

■ Heterogeneity
Nodes may run different OSes, languages, hardware. Use standard protocols (HTTP, Protobuf).

1.4 Types of Distributed Systems

High-Performance Computing (HPC) Clusters
Tightly coupled nodes connected via high-speed InfiniBand. Used for weather simulation, molecular dynamics. **Example:** Supercomputer Fugaku (Japan) with 7.6 million cores solving climate models.

Distributed Information Systems
Transaction-processing systems and enterprise integration middleware. **Example:** Banking SWIFT network transferring $5 trillion daily across 11,000+ institutions.

Pervasive / IoT Systems
Embedded, mobile, resource-constrained nodes. **Example:** Smart home — thermostat, locks, cameras collaborating via MQTT broker with cloud backend.

Cloud Computing Systems
Virtualised, elastically scalable infrastructure. AWS, Azure, GCP offer IaaS/PaaS/SaaS. Users pay for what they use; providers handle hardware.

> **■ Key Insight**
>
> The fundamental impossibility results — FLP Impossibility, CAP Theorem — shape every design decision in distributed systems. Understanding *what cannot be achieved* is just as important as knowing what can.

CHAPTER 2

Communication in Distributed Systems

RPC · Message Queues · Pub-Sub · REST vs gRPC

2.1 Remote Procedure Call (RPC) & gRPC

RPC lets a client call a function on a remote server as if it were local. The **stub** (client-side) serialises arguments into a network message; the **skeleton** (server-side) deserialises and invokes the actual function.

gRPC (Google RPC)

gRPC uses **Protocol Buffers (Protobuf)** for serialisation and **HTTP/2** for transport. Benefits: strongly typed contracts, bi-directional streaming, efficient binary encoding (~5-10× smaller than JSON), built-in TLS.

gRPC Service Definition (user.proto)

```
syntax = "proto3";

service UserService {
rpc GetUser (UserRequest) returns (UserResponse);
rpc ListUsers (ListRequest) returns (stream UserResponse); // server-streaming
}

message UserRequest { string user_id = 1; }
message UserResponse { string name = 1; string email = 2; int32 age = 3; }
```

Python gRPC Server (generated stub)

```
import grpc, user_pb2, user_pb2_grpc

class UserServicer(user_pb2_grpc.UserServiceServicer):
def GetUser(self, request, context):
user = db.find(request.user_id) # local DB call
return user_pb2.UserResponse(
name=user.name, email=user.email, age=user.age)

server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
user_pb2_grpc.add_UserServiceServicer_to_server(UserServicer(), server)
server.add_insecure_port("[::]:50051")
server.start(); server.wait_for_termination()
```

2.2 Message Queues

Message queues provide **asynchronous, decoupled** communication. Producers push messages into a queue; consumers pull and process at their own pace. This prevents fast producers from overwhelming slow consumers (*backpressure*).

■ Example: Order Processing with RabbitMQ

An e-commerce checkout service puts an 'order-placed' message on a queue. Three independent consumers — inventory, payment, notification — each read from their own queue copy (exchange → binding). If the payment service is slow or crashes, messages accumulate safely until it recovers.

Python — Publish to RabbitMQ (pika)

```
import pika, json

connection = pika.BlockingConnection(
pika.ConnectionParameters("rabbitmq-host"))
channel = connection.channel()
channel.queue_declare(queue="orders", durable=True)

order = {'id': 'ORD-1234', 'item': 'Laptop', 'qty': 1}
channel.basic_publish(
exchange="", routing_key="orders",
body=json.dumps(order),
properties=pika.BasicProperties(delivery_mode=2)) # persistent
connection.close()
```

2.3 Publish-Subscribe Pattern

In pub-sub, publishers emit **events to topics** without knowing who will receive them. Subscribers express interest in topics. A **broker** (Kafka, Google Pub/Sub, Redis Streams) routes events. This enables fan-out to many consumers and event-driven architectures.

Apache Kafka Deep Dive

Kafka organises messages in **topics** → **partitions** → **offsets**. Each partition is an ordered, immutable log. Consumers track their offset, allowing replay. Kafka retains messages for days/weeks (not just until consumed).

Kafka Guarantees

- At-least-once delivery by default (consumer may reprocess on crash)
- Exactly-once with idempotent producers + transactions (Kafka 0.11+)
- Ordering guaranteed within a partition, not across partitions
- Throughput: millions of messages/sec on commodity hardware

2.4 REST vs GraphQL vs gRPC — Comparison

Feature	REST	gRPC
Protocol	HTTP/1.1 or 2	HTTP/2
Format	JSON / XML	Protobuf (binary)
Typing	No schema enforcement	Strongly typed (.proto)
Streaming	No (SSE workaround)	Bidirectional native
Best For	Public APIs, browser	Internal microservices
Tooling	Universal	Generated stubs

CHAPTER 3

Time, Clocks & Ordering

Physical Clocks · Logical Clocks · Vector Clocks

3.1 Physical Clocks & The NTP Problem

Physical clocks use quartz oscillators that drift ≈ 1 second per day. **NTP (Network Time Protocol)** synchronises clocks over the internet to within ~50 ms accuracy (≈1 ms on LANs). Google's **TrueTime API** (Spanner) exposes clock uncertainty as an interval [earliest, latest] and waits out uncertainty before committing.

Why Physical Time Fails for Ordering

Node A records event e1 at t=1000ms. Node B records e2 at t=999ms. Network lag means B's clock is behind. We cannot conclude e2 happened before e1 based on wall-clock timestamps alone. We need logical ordering.

3.2 Lamport Logical Clocks

Leslie Lamport (1978) defined the **happens-before (→)** relation: event a → b if they are in the same process with a before b, or a sends a message received by b. Lamport clocks assign each event a timestamp satisfying: if a → b then L(a) < L(b).

Algorithm

- Each process maintains counter **C**, initially 0.
- On any local event: C = C + 1
- On sending message: C = C + 1; attach C to message
- On receiving message m with timestamp T: C = max(C, T) + 1

Lamport Clock — Python Implementation

```
class LamportClock:
def __init__(self): self.time = 0

def tick(self): # local event
self.time += 1
return self.time

def send(self): # before sending
self.time += 1
return self.time # attach to message

def receive(self, msg_time): # on receiving
```

```
    self.time = max(self.time, msg_time) + 1
    return self.time
```

3.3 Vector Clocks

Lamport clocks detect causality but not concurrency. **Vector clocks** store one counter per process. VCa < VCb iff every entry of VCa ≤ VCb (strict for at least one). If neither V(a) ≤ V(b) nor V(b) ≤ V(a), the events are **concurrent**.

■ Example: Dynamo DB uses vector clocks to detect write conflicts

When two clients update the same key on two replicas without seeing each other's write, their vector clocks diverge. On the next read, the client receives both versions as a conflict and must reconcile them (like a shopping cart merge).

Vector Clock — Python

```
class VectorClock:
    def __init__(self, pid, n):
        self.pid = pid # this process id (0-indexed)
        self.vc = [0] * n # n = total processes

    def local_event(self):
        self.vc[self.pid] += 1

    def send_event(self):
        self.vc[self.pid] += 1
        return list(self.vc) # send copy

    def receive_event(self, recv_vc):
        for i in range(len(self.vc)):
            self.vc[i] = max(self.vc[i], recv_vc[i])
        self.vc[self.pid] += 1

    def concurrent(self, other_vc):
        le = all(self.vc[i] <= other_vc[i] for i in range(len(self.vc)))
        ge = all(self.vc[i] >= other_vc[i] for i in range(len(self.vc)))
        return not le and not ge # neither dominates
```

CHAPTER 4

Consistency & Replication

Consistency Models · Replication Strategies · Quorums · CRDTs

4.1 Consistency Models (Strongest to Weakest)

Linearizability — (Strong Consistency)

Every operation appears to take effect instantaneously at some point between its invocation and response. Example: Zookeeper znodes, Google Spanner. Cost: high latency.

Sequential Consistency

All nodes see operations in the same order, but that order need not match real time. Example: single-master SQL replication.

Causal Consistency

Causally related operations are seen in order by all nodes. Concurrent ops may differ. Example: MongoDB causally-consistent sessions.

Read-Your-Writes

After a write, the same client always sees its write on subsequent reads. Example: User profile update immediately visible to the user.

Monotonic Read

Once a client reads a value, subsequent reads never return older values. Example: social media feed — you don't see old posts after seeing newer ones.

Eventual Consistency

If no new writes occur, all replicas eventually converge. Example: Amazon DynamoDB, DNS propagation, Cassandra.

4.2 Replication Strategies

Single-Leader (Primary-Replica)

All writes go to the **leader**; the leader replicates to followers. Reads can be served by followers (may be stale) or the leader (consistent). **Synchronous replication**: leader waits for follower ACK → no data loss, but higher write latency. **Asynchronous**: leader doesn't wait → low latency but potential data loss on crash.

■ Example: PostgreSQL Streaming Replication

Primary streams WAL (Write-Ahead Log) segments to standbys. Standbys replay WAL to stay in sync. On primary failure, one standby is promoted. Replication lag monitoring via pg_stat_replication.

Multi-Leader Replication

Multiple nodes accept writes. Useful for multi-datacenter deployments (one leader per DC). Writes in different DCs are replicated asynchronously. **Write conflicts** must be resolved (last-write-wins, CRDTs, application logic). **Example:** Google Docs uses OT (Operational Transformation) for conflict resolution.

Leaderless (Dynamo-style)

Any replica accepts writes. Client writes to W replicas; reads from R replicas. Reads perform **read repair** to fix stale replicas. **Example:** Cassandra, Riak, Amazon DynamoDB.

4.3 Quorum Systems

With N replicas, write quorum W, read quorum R: if W + R > N, every read set and write set overlap → reads always see the latest write.

Quorum Formula: W + R > N

- N=3, W=2, R=2: tolerates 1 replica failure. Most common setup.
- N=5, W=3, R=3: tolerates 2 failures. Used by Cassandra RF=5.
- N=3, W=1, R=3: write-optimised (fast writes, slow reads).
- N=3, W=3, R=1: read-optimised (slow writes, fast reads).

4.4 CRDTs (Conflict-free Replicated Data Types)

CRDTs are data structures that can be updated concurrently on multiple replicas and merged automatically without conflicts. They guarantee eventual consistency mathematically.

G-Counter (Grow-only Counter) — Example

G-Counter CRDT Implementation

```
class GCounter:
def __init__(self, node_id, n): # n = cluster size
self.node_id = node_id
self.counts = [0] * n

def increment(self):
self.counts[self.node_id] += 1

def value(self):
return sum(self.counts)

def merge(self, other): # commutative, associative, idempotent
for i in range(len(self.counts)):
self.counts[i] = max(self.counts[i], other.counts[i])
```

Real-world use: Redis CRDT module, Riak's distributed counters, shopping cart totals in Amazon Dynamo paper.

CHAPTER 5

Consensus Algorithms

Paxos · Raft · ZAB · FLP Impossibility

5.1 The Consensus Problem

Given N processes, each proposing a value, consensus requires: **Agreement** (all correct processes decide the same value), **Validity** (the decided value was proposed by some process), and **Termination** (all correct processes eventually decide).

FLP Impossibility (Fischer, Lynch, Paterson — 1985)

In a purely asynchronous system with even ONE possible crash failure, no deterministic consensus algorithm can guarantee termination. Practical systems escape this by using timeouts (randomness or partial synchrony).

5.2 Paxos

Paxos (Lamport, 1989) is the seminal consensus protocol. It tolerates up to ■(N-1)/2■ crash failures (not Byzantine). It operates in two phases:

Phase 1 — Prepare / Promise

- Proposer selects unique proposal number n and sends Prepare(n) to quorum.
- Acceptor promises to ignore requests with number < n; returns highest previously accepted value.

Phase 2 — Accept / Accepted

- Proposer sends Accept(n, v) where v is highest returned value (or own value if none).
- Acceptor accepts if no prepare(n') with n' > n was received; notifies learners.

Multi-Paxos skips Phase 1 for subsequent values once a stable leader is elected, reducing round trips. Used in Chubby (Google), Zookeeper, Cassandra (lightweight transactions).

5.3 Raft

Raft (Ongaro & Ousterhout, 2014) was designed for *understandability*. It decomposes consensus into three sub-problems: leader election, log replication, and safety.

Leader Election

Nodes start as **Followers**. If no heartbeat within **election timeout** (150–300ms random), a follower becomes a **Candidate**, increments its term, votes for itself, and requests votes. A node receiving RequestVote grants it if term ≥ current term and it hasn't voted. Majority wins → becomes **Leader**.

Log Replication

Client sends command to leader. Leader appends to its log, sends AppendEntries RPCs to followers. Once a majority ACK, the entry is **committed**. Leader applies to state machine and responds to client. Followers apply on next heartbeat.

■ Example: etcd — Kubernetes uses Raft via etcd

etcd stores cluster state (pod specs, secrets). A 3-node etcd cluster tolerates 1 failure (needs 2 for quorum). All writes go through the Raft leader. etcd recommends SSD-backed storage as disk flush is on the critical path.

5.4 ZAB & Apache Zookeeper

ZAB (Zookeeper Atomic Broadcast) is similar to Paxos but optimised for primary-backup architectures. Zookeeper provides coordination primitives: ephemeral nodes (deleted when session ends), sequential nodes, and **watches** (one-time notification of changes). Used for distributed locks, leader election, service discovery (before etcd).

CHAPTER 6

Fault Tolerance & Reliability

Failure Models · Circuit Breakers · Byzantine Faults

6.1 Failure Models

Failure Type	Description
Crash-Stop	Node stops and never recovers. Simplest model. Detectable via timeout.
Crash-Recovery	Node may crash and restart with durable state intact. Most real systems.
Omission	Node fails to send/receive some messages. Simulates network packet loss.
Timing / Performance	Node is slow — responds outside expected time window.
Byzantine	Node behaves arbitrarily: sends wrong data, colludes, lies. Hardest model. Requires ≥ 3f+1 nodes to tolerate f Byzantine faults.

6.2 Circuit Breaker Pattern

A circuit breaker wraps remote calls. It monitors failures; if errors exceed a threshold within a window, it **opens** the circuit and fast-fails subsequent calls (returning cached/default responses). After a timeout, it transitions to **half-open** and tests one request. On success → closed; on failure → open again.

Circuit Breaker — Python (simplified)

```
import time

class CircuitBreaker:
CLOSED, OPEN, HALF_OPEN = 'CLOSED', 'OPEN', 'HALF_OPEN'

def __init__(self, threshold=5, timeout=60):
self.state = self.CLOSED
self.failures = 0
self.threshold = threshold
self.timeout = timeout
self.last_fail = None

def call(self, func, *args):
if self.state == self.OPEN:
```

```
    if time.time() - self.last_fail > self.timeout:
        self.state = self.HALF_OPEN
    else:
        raise Exception('Circuit OPEN — fast fail')
    try:
        result = func(*args)
        self._success()
        return result
    except Exception:
        self._fail(); raise

def _success(self):
    self.failures = 0; self.state = self.CLOSED

def _fail(self):
    self.failures += 1; self.last_fail = time.time()
    if self.failures >= self.threshold:
        self.state = self.OPEN
```

Netflix Hystrix (now Resilience4j) popularised circuit breakers in microservices.

6.3 Byzantine Fault Tolerance (BFT)

PBFT (Practical Byzantine Fault Tolerance — Castro & Liskov, 1999) tolerates f Byzantine nodes with 3f+1 total nodes in a 3-phase protocol: Pre-Prepare, Prepare, Commit. Used in blockchain systems. **Bitcoin/Ethereum** use Proof-of-Work as a Sybil-resistant BFT mechanism. **Hyperledger Fabric** uses PBFT variants for permissioned blockchains.

CHAPTER 7

CAP Theorem & PACELC

Consistency · Availability · Partition Tolerance

7.1 CAP Theorem (Brewer, 2000; Proof: Gilbert & Lynch, 2002)

In a distributed system, it is **impossible** to simultaneously guarantee all three of: **Consistency** (every read receives the most recent write or an error), **Availability** (every request receives a non-error response, though possibly stale), **Partition Tolerance** (system continues operating despite network partitions).

Since network partitions are a reality (not optional), designers must choose between **CP** (sacrifice availability during partitions) or **AP** (sacrifice consistency).

Type	Behaviour	Examples
CA	No partition tolerance (single node)	Traditional RDBMS, LDAP
CP	Consistent; unavailable during partition	HBase, Zookeeper, etcd, MongoDB(default)
AP	Available + eventually consistent	Cassandra, DynamoDB, CouchDB, DNS

7.2 PACELC — A More Complete Model (Abadi, 2012)

CAP only addresses behaviour during partitions. PACELC adds: even in the absence of partitions (E = else), there is a tradeoff between **Latency (L)** and **Consistency (C)**.

PACELC Formula

If Partition: choose between Availability vs Consistency

Else (normal ops): choose between Latency vs Consistency

Example: Cassandra is PA/EL — available on partition, low latency normally (tunable consistency). Google Spanner is PC/EC — consistent always, higher latency.

7.3 Real-World CAP Tradeoffs

Banking System (CP)

A bank cannot show a wrong balance. During a network partition, ATMs may go offline (unavailable) rather than allow withdrawals that could overdraw accounts. Consistency > Availability.

Shopping Cart (AP)

Amazon Dynamo paper: it's better to show a slightly stale cart (with stale item) than to error out. The worst case is adding an item twice — easily reconciled. Availability > Consistency.

Social Media Feed (AP)

A tweet visible to some users before others is acceptable. Eventual consistency at global scale. Twitter uses Cassandra for timelines.

CHAPTER 8

Distributed Transactions

ACID vs BASE · 2PC · Saga · Distributed Locks

8.1 ACID vs BASE

ACID (Traditional RDBMS)	BASE (Distributed NoSQL)
Atomicity: all-or-nothing	Basically Available: system available most of the time
Consistency: DB invariants always hold	Soft State: state may change without input (replicas converging)
Isolation: concurrent txns see committed state	Eventually Consistent: converges given no new writes
Durability: committed data survives crashes	Optimistic conflict resolution; high availability priority

8.2 Two-Phase Commit (2PC)

2PC coordinates an atomic transaction across multiple databases/services using a **coordinator** and **participants**.

Phase 1 — Voting

- Coordinator sends PREPARE to all participants.
- Each participant executes transaction, logs to WAL, votes YES or NO.

Phase 2 — Commit/Abort

- If all YES → coordinator sends COMMIT; participants apply and release locks.
- If any NO → coordinator sends ABORT; participants rollback.

2PC Problem: Blocking Protocol

If the coordinator crashes after sending PREPARE but before COMMIT, participants are **blocked** holding locks indefinitely. 3PC and Paxos-based commit solve this at higher complexity.

8.3 Saga Pattern

A Saga is a sequence of local transactions, each publishing an event that triggers the next. On failure, **compensating transactions** undo previously completed steps (like a refund for a charged order).

■ Example: E-commerce Order — Choreography Saga

1. OrderService creates order → publishes OrderCreated. 2. PaymentService charges card → publishes PaymentProcessed. 3. InventoryService reserves stock → publishes StockReserved. 4. ShipmentService ships → publishes OrderShipped.

If Step 3 fails (out of stock): InventoryService publishes StockFailed → PaymentService listens and issues refund → OrderService marks order FAILED.

8.4 Distributed Locks

Distributed locks prevent concurrent access to shared resources across nodes. Implemented with: **Redis SETNX** (set if not exists) with TTL expiry, **Zookeeper ephemeral sequential nodes**, or **database advisory locks**.

Redis Distributed Lock (Redlock concept)

```
import redis, uuid, time

def acquire_lock(client, lock_name, ttl_ms=5000):
token = str(uuid.uuid4())
acquired = client.set(lock_name, token,
px=ttl_ms, nx=True) # NX = only if not exists
return token if acquired else None

def release_lock(client, lock_name, token):
# Lua script ensures atomic check-and-delete
lua = '''
if redis.call('get',KEYS[1]) == ARGV[1] then
return redis.call('del',KEYS[1])
else return 0 end'''
client.eval(lua, 1, lock_name, token)
```

CHAPTER 9

Load Balancing & Scalability

Algorithms · Consistent Hashing · Sharding · Auto-Scaling

9.1 Load Balancing Algorithms

Round Robin

Requests distributed cyclically. Simple; ignores server load. Best when servers are homogeneous.

Weighted Round Robin

Servers assigned weights proportional to capacity. More requests to powerful nodes.

Least Connections

Route to server with fewest active connections. Good for variable-length requests.

IP Hash

Hash client IP → same server always. Provides session stickiness. Problem: hot spots.

Random

Random server selection. Surprises can be good; loses locality.

Resource-Based

Route based on actual CPU/memory of servers (requires health-check metrics). Most intelligent.

9.2 Horizontal vs Vertical Scaling

Horizontal (Scale Out)	Vertical (Scale Up)
Add more nodes (cheaper commodity hardware)	Upgrade existing node (more CPU/RAM/SSD)
Fault tolerant by design (redundancy)	Single point of failure
Requires distributed system design (partitioning)	Simpler operationally; no data distribution needed
Examples: Cassandra, Kafka, HDFS	Examples: PostgreSQL on large EC2, Oracle RAC

9.3 Consistent Hashing

In a ring of hash values (0 to 2^{32}-1), both nodes and keys are hashed onto the ring. A key is served by the first node clockwise from its position. Adding/removing a node only remaps keys between the new node and its predecessor — minimising data movement (unlike modulo hashing).

■ Example: Cassandra & Consistent Hashing

Cassandra uses a variant called **Virtual Nodes (vnodes)**. Each physical node owns multiple small token ranges (typically 256). This spreads load evenly and makes adding nodes gradual. When a node is added, each existing node donates a small portion of its ranges.

Consistent Hashing Ring — Python

```
import hashlib, bisect

class ConsistentHash:
def __init__(self, replicas=150): # virtual nodes per server
self.replicas = replicas
self.ring = {}; self.sorted_keys = []

def _hash(self, key):
return int(hashlib.md5(key.encode()).hexdigest(), 16)

def add_node(self, node):
for i in range(self.replicas):
h = self._hash(f'{node}-{i}')
self.ring[h] = node
bisect.insort(self.sorted_keys, h)

def get_node(self, key):
h = self._hash(key)
idx = bisect.bisect(self.sorted_keys, h) % len(self.sorted_keys)
return self.ring[self.sorted_keys[idx]]
```

9.4 Database Sharding

Sharding partitions a database horizontally across multiple machines. Each shard holds a subset of rows. Common strategies:

- **Range-based:** Shard by range of key values (e.g., user IDs 1–1M on shard 1). Risk: hot shards if access is skewed.
- **Hash-based:** Shard = hash(key) % N. Uniform distribution but range queries span shards.
- **Directory-based:** Lookup table maps keys to shards. Flexible but lookup is a bottleneck.
- **Geo-based:** Users in Europe → European shard. Reduces latency; mandated by GDPR.

CHAPTER 10

Distributed Storage

DFS · NoSQL · NewSQL · Data Partitioning

10.1 Distributed File Systems

HDFS (Hadoop Distributed File System)

HDFS splits large files into **128 MB blocks** replicated 3× across DataNodes. The **NameNode** stores metadata (file tree, block locations). Designed for batch processing (write-once, read-many). Used by Hive, Spark, HBase as underlying storage.

Google GFS / Colossus

GFS (2003 paper) inspired HDFS. Master node + chunk servers (64 MB chunks). Colossus (GFS2) uses distributed metadata management to eliminate single-master bottleneck. Underpins Bigtable, Spanner, and Google Search index.

10.2 NoSQL Databases

Key-Value Stores — Redis

In-memory data structure store. O(1) GET/SET. Supports strings, hashes, lists, sorted sets, bitmaps, HyperLogLog. Persistence via RDB snapshots and AOF log. Cluster mode: 16384 hash slots across nodes. Used for caching, sessions, rate limiting.

Wide-Column — Apache Cassandra

Cassandra organises data in tables with a **Partition Key** (determines which node) and **Clustering Columns** (ordering within partition). No single point of failure. Write path: CommitLog → Memtable → SSTable flush. Compaction merges SSTables and removes tombstones (deleted rows).

Cassandra CQL — Table Design

```
CREATE TABLE messages (
conversation_id UUID,
sent_at TIMESTAMP,
sender_id UUID,
body TEXT,
PRIMARY KEY (conversation_id, sent_at) -- partition + clustering
) WITH CLUSTERING ORDER BY (sent_at DESC);

-- Fetches all messages in a conversation, newest first
SELECT * FROM messages WHERE conversation_id = ?
AND sent_at > '2024-01-01' LIMIT 50;
```

Document Store — MongoDB

Stores JSON-like BSON documents. Schema-less (flexible fields per document). Rich query language, secondary indexes, aggregation pipelines. Replica sets for HA; sharded clusters for horizontal scale. WiredTiger storage engine with compression and MVCC.

Graph Database — Neo4j

Stores nodes and relationships natively. Ideal for social graphs, recommendation engines, fraud detection. Cypher query language. Example: LinkedIn uses graph DB for '2nd-degree connections'.

10.3 NewSQL — Google Spanner

Spanner provides ACID transactions at global scale using **TrueTime** (GPS + atomic clocks) to assign globally consistent timestamps. Paxos-replicated across zones; SQL interface with external consistency. F1 (Google Ads) migrated from sharded MySQL to Spanner, eliminating manual resharding at cost of higher latency.

CHAPTER 11

Microservices & Service Mesh

Architecture · Discovery · API Gateway · Istio

11.1 Microservices Architecture

Microservices decompose a monolith into small, independently deployable services, each owning its data and communicating over APIs. Benefits: independent scaling, technology heterogeneity, fault isolation, small teams. Costs: network overhead, distributed transactions complexity, observability difficulty.

■ Example: Netflix Microservices

Netflix runs 700+ microservices. Each team owns their service end-to-end (you build it, you run it). Chaos Monkey randomly kills services in production to ensure resilience. Each service can scale independently — recommendation engine scales separately from billing.

11.2 Service Discovery

Services need to find each other without hardcoded IPs. Two patterns:

Client-Side Discovery

Client queries a **service registry** (Consul, Eureka) for available instances, then applies a load-balancing algorithm itself. Netflix Ribbon does this. Pro: client controls routing logic. Con: client must know registry.

Server-Side Discovery

Client calls a **load balancer**; the LB queries the registry. AWS ALB + ECS does this. Client is unaware of backend topology. Con: LB is another network hop.

11.3 API Gateway

An API Gateway is the single entry point for all clients. It handles: authentication/authorisation, SSL termination, rate limiting, request routing, response aggregation, logging. Examples: Kong, AWS API Gateway, Nginx, Traefik.

BFF — Backend for Frontend

A variant where each client type (mobile, web, third-party) gets its own gateway that aggregates and transforms responses optimised for that client. Reduces over-fetching and simplifies client code.

11.4 Service Mesh — Istio

A service mesh adds a **sidecar proxy** (Envoy) to every pod. The data plane (proxies) handles mTLS, retries, timeouts, circuit breaking, and telemetry — transparently, without code changes. The control plane (Istio Pilot, Citadel) configures proxies.

- **Traffic Management:** Canary deployments, A/B testing, blue-green via VirtualService rules.
- **Security:** Automatic mutual TLS between all services; RBAC policies.
- **Observability:** Automatic distributed tracing (Jaeger), metrics (Prometheus), access logs.

CHAPTER 12

Observability & Monitoring

Tracing · Metrics · Logging · Alerting

12.1 The Three Pillars of Observability

- **Metrics:** Numerical measurements over time (request rate, latency, error rate, CPU). Time-series databases: Prometheus, InfluxDB.
- **Logs:** Timestamped event records. Structured logs (JSON) enable querying. ELK stack (Elasticsearch, Logstash, Kibana) or Loki.
- **Traces:** Record the journey of a request across services. Each operation is a **span**; spans form a **trace tree**.

12.2 Distributed Tracing

Each request is assigned a unique **trace ID**. Each service creates a **span** with: trace ID, span ID, parent span ID, operation name, start/end time, tags. Trace context propagates via HTTP headers (W3C TraceContext standard: traceparent). Jaeger and Zipkin collect and visualise traces.

OpenTelemetry Tracing — Python

```
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.exporter.jaeger.thrift import JaegerExporter

provider = TracerProvider()
exporter = JaegerExporter(agent_host_name='jaeger', agent_port=6831)
provider.add_span_processor(BatchSpanProcessor(exporter))
trace.set_tracer_provider(provider)
tracer = trace.get_tracer('order-service')

def process_order(order_id):
with tracer.start_as_current_span('process_order') as span:
span.set_attribute('order.id', order_id)
result = db.fetch_order(order_id) # auto-creates child span
span.set_attribute('order.status', result.status)
return result
```

12.3 The RED Method for Services

Monitor every service with three golden signals:

- **R**ate — requests per second (throughput)
- **E**rror — error rate (% of requests failing)
- **D**uration — latency distribution (p50, p95, p99)

Prometheus query examples: rate(http_requests_total[5m]) for rate; histogram_quantile(0.99, rate(http_duration_seconds_bucket[5m])) for p99 latency.

12.4 Health Checks & Alerting

Health Check Endpoints

Services expose /health (liveness: is the process alive?) and /ready (readiness: is it ready to serve traffic?). Kubernetes uses these to restart crashed pods and stop routing to pods still warming up.

SLO-Based Alerting

SLI (Service Level Indicator): measured metric (e.g., 99.5% of requests < 200ms). **SLO** (Objective): target (e.g., 99.9% availability per month = 43 min downtime). **SLA** (Agreement): contractual SLO with penalties. **Error Budget**: allowed failures before SLO breach. Alert when error budget burn rate exceeds threshold (e.g., 2× for 1h or 5× for 5min → page on-call).

END OF DISTRIBUTED SYSTEMS COMPLETE REFERENCE GUIDE

Chapters covered: Fundamentals · Communication · Clocks · Consistency · Consensus · Fault Tolerance · CAP/PACELC · Transactions · Scalability · Storage · Microservices · Observability